

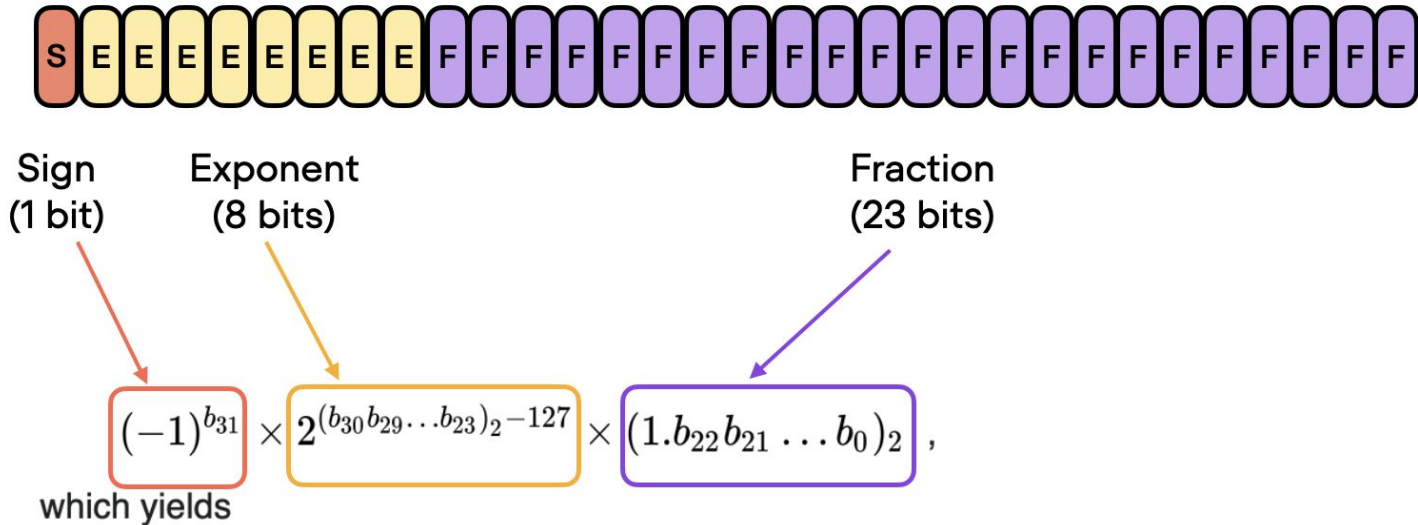
LSDL Seminar 03

Efficient training and inference

Ildus Sadrtidinov, 22.09.2025

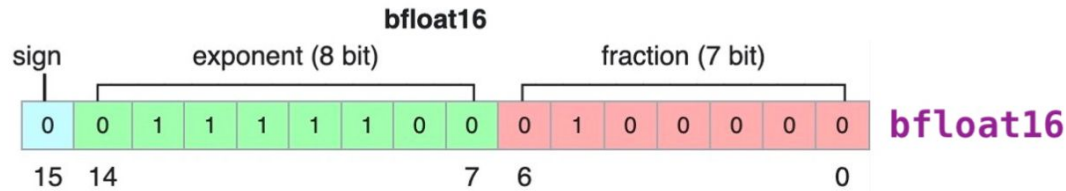
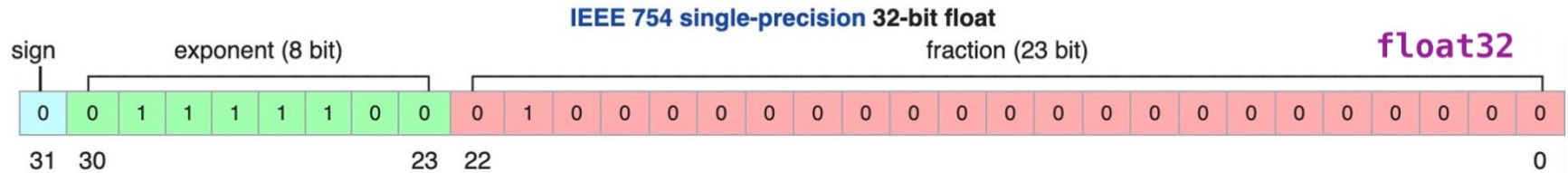
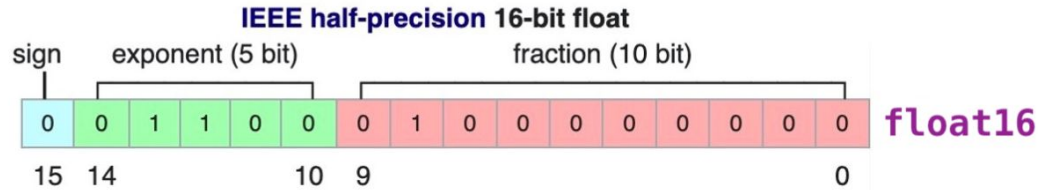
Floating point numbers

float 32



$$\text{value} = (-1)^{\text{sign}} \times 2^{(E-127)} \times \left(1 + \sum_{i=1}^{23} b_{23-i} 2^{-i} \right).$$

Floating point formats

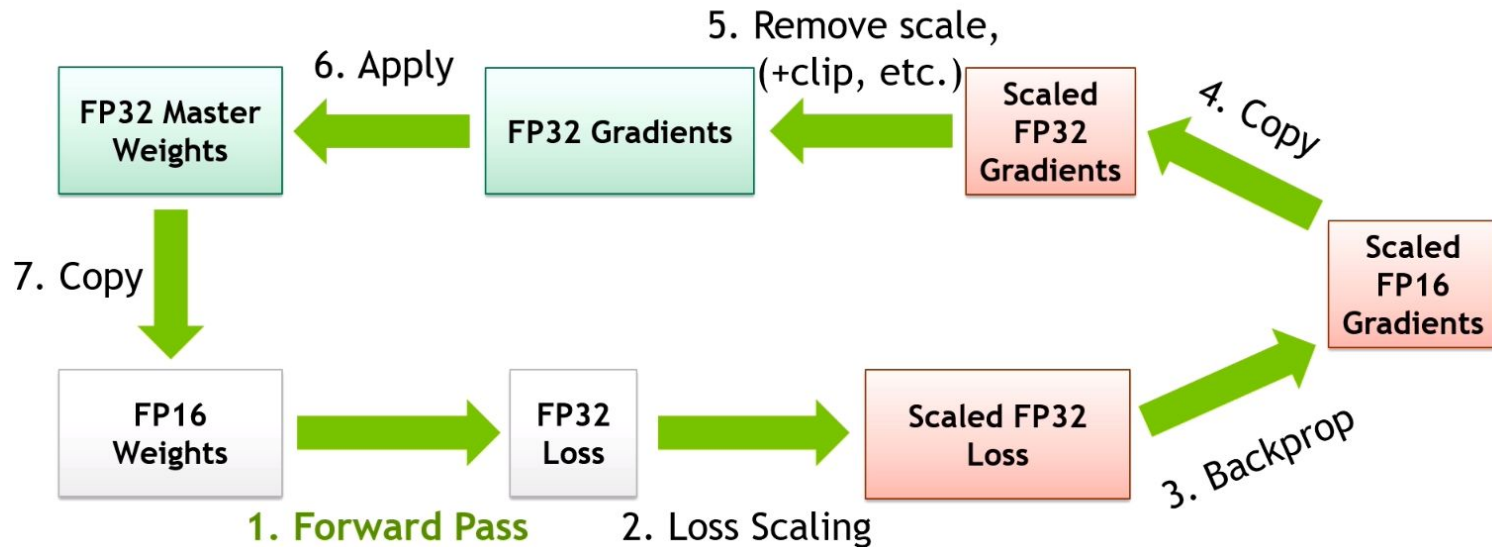


Automatic Mixed Precision (AMP)

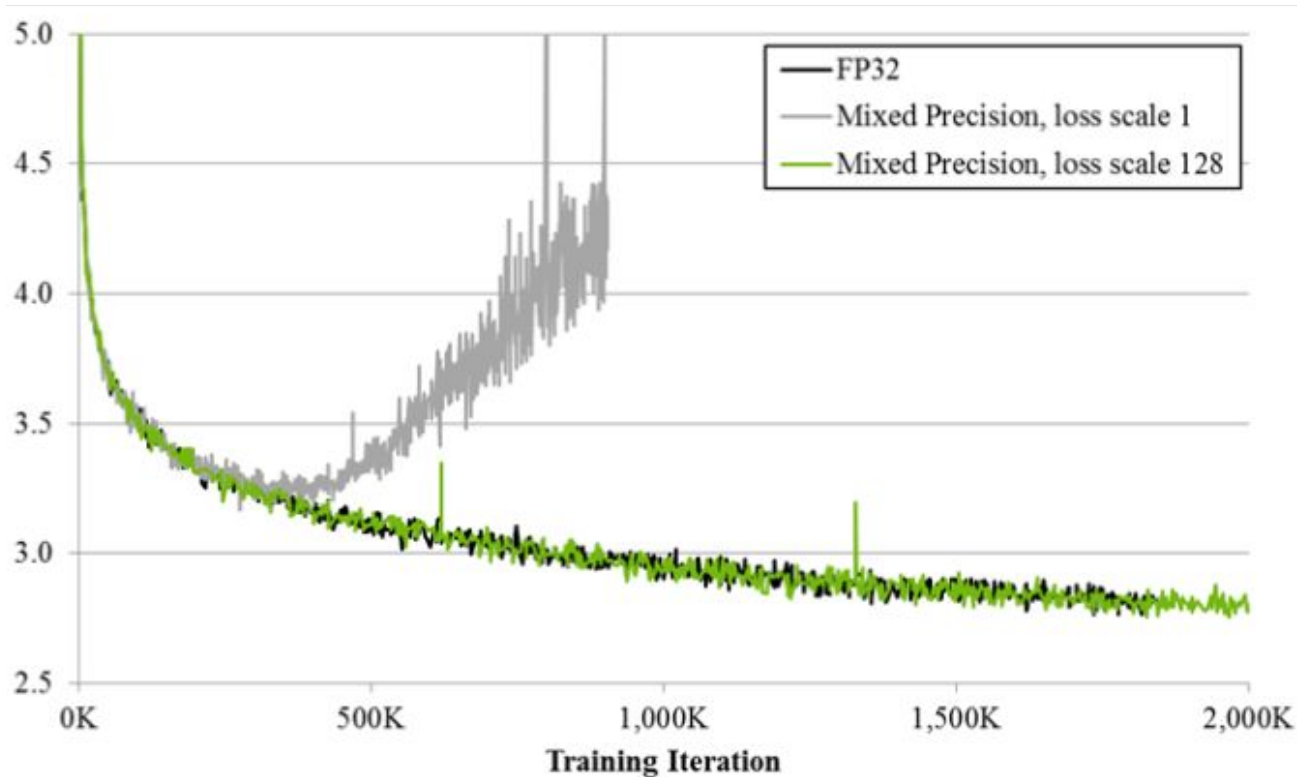
- Training in **FP16** diverges
- Some operations are feasible with **FP16** (matrix multiplication), while other require **FP32** (softmax, normalization)
- Data cast gives almost no overhead
- We can use **FP16** where appropriate to speed up forward and backward passes
- Loss scaling is required

Automatic Mixed Precision (AMP)

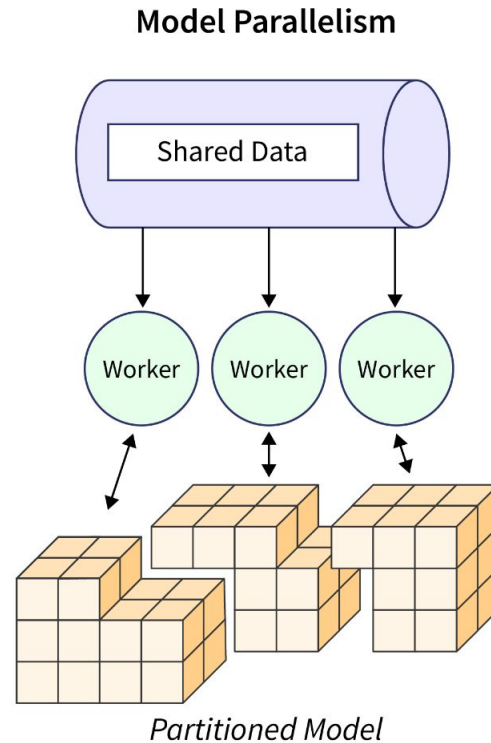
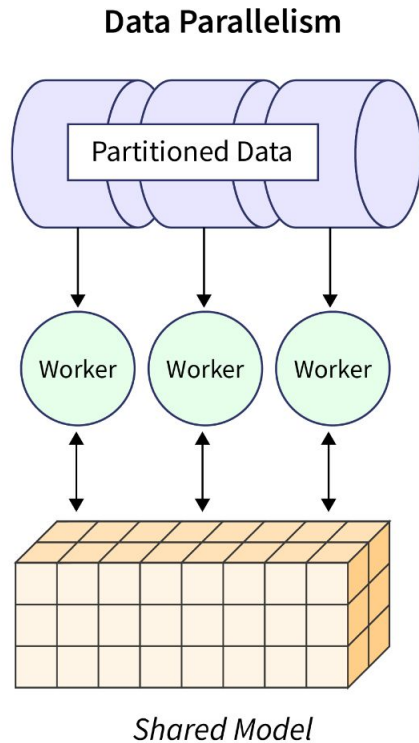
MIXED PRECISION TRAINING



Automatic Mixed Precision

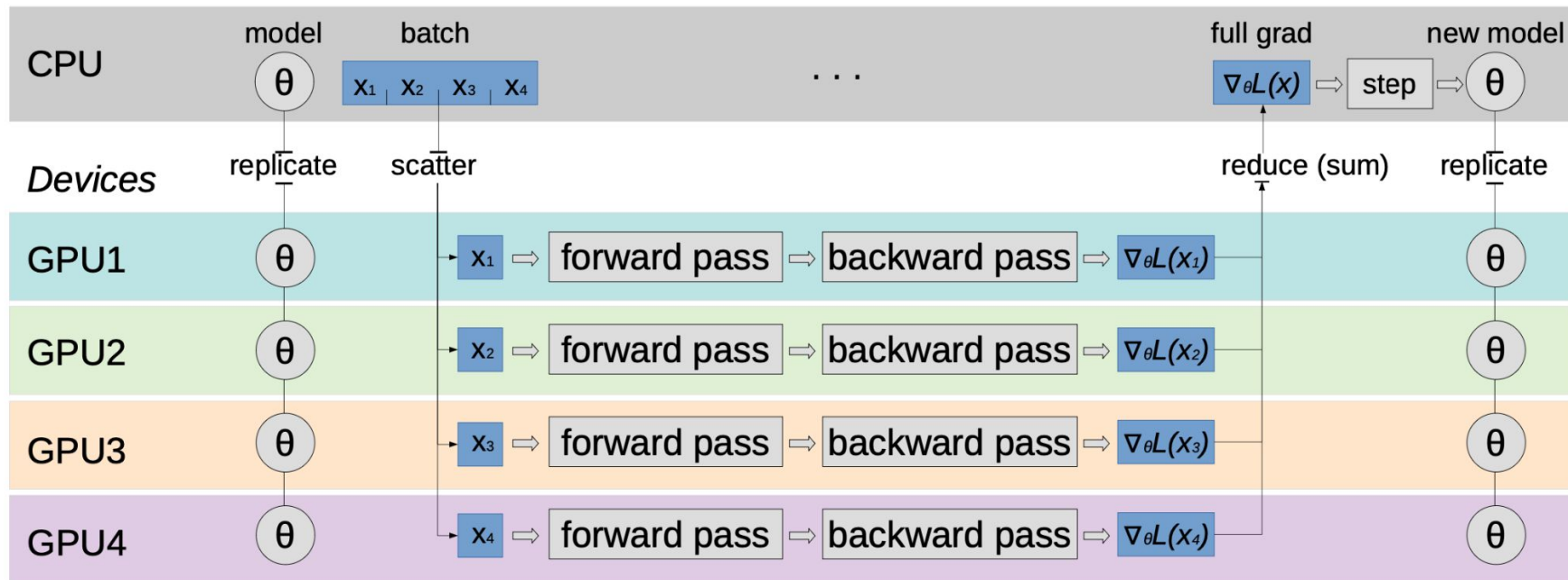


Two paradigms of parallelism



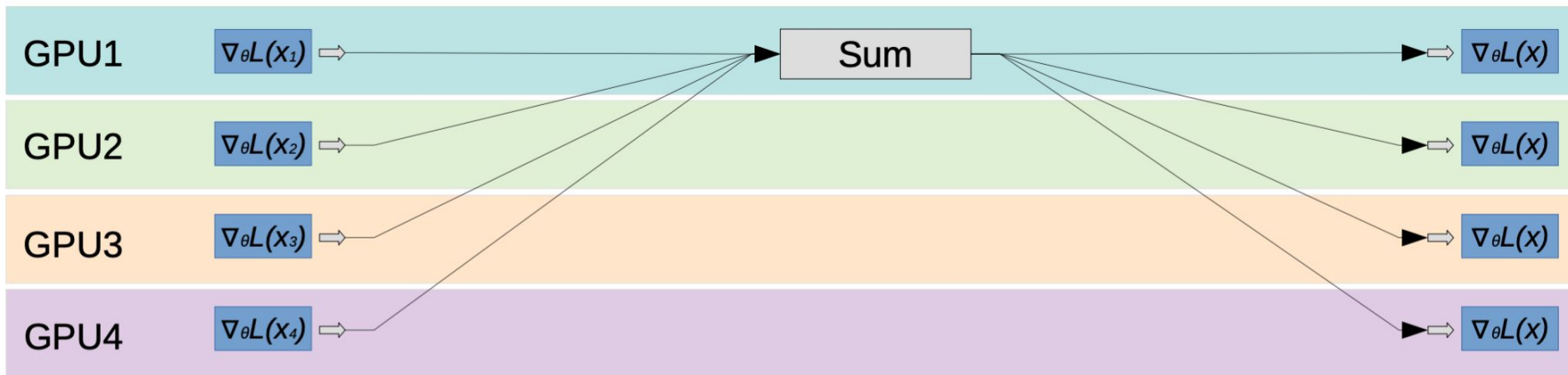
Data Parallelism

Host



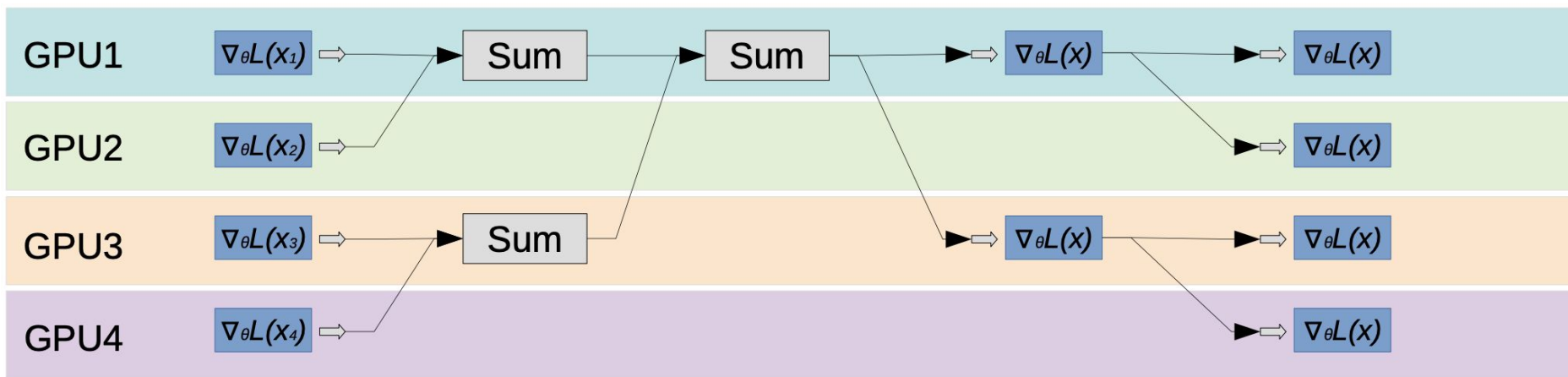
Naive All-Reduce

Devices



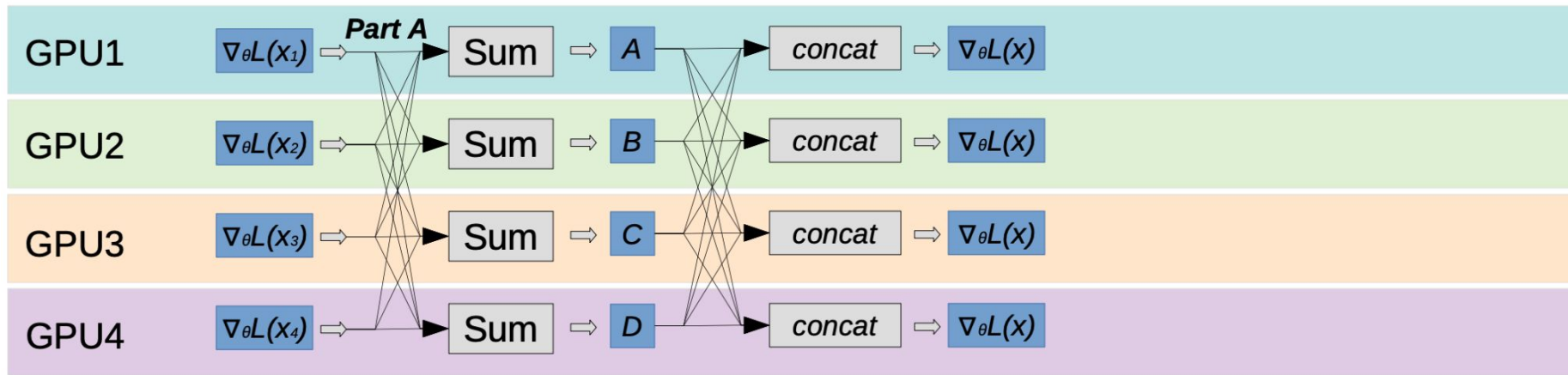
Tree All-Reduce

Devices

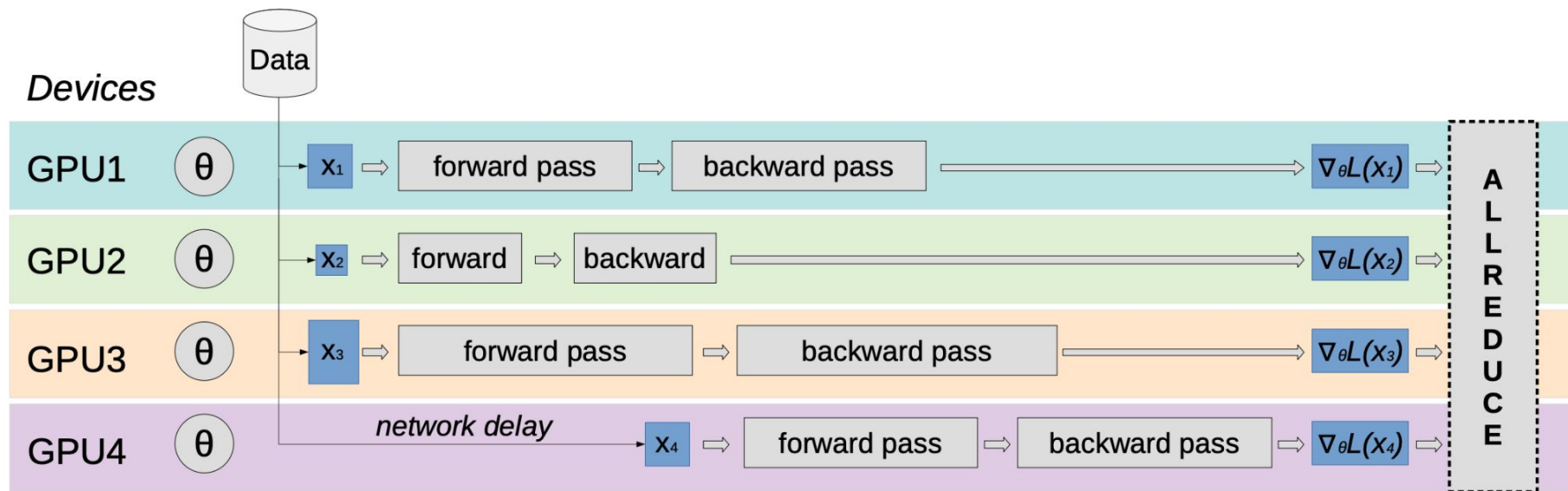


Butterfly All-Reduce

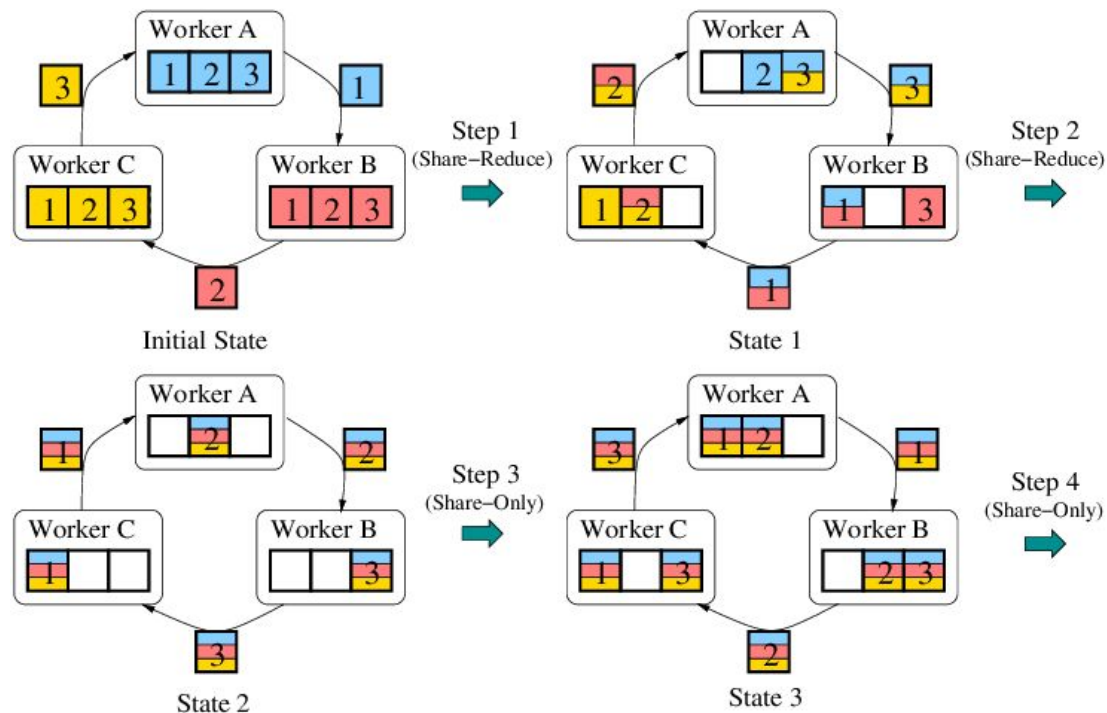
Devices



What happens in practice?



Ring All-Reduce



Gradient subvector n of Worker A ($n = 1, 2, 3$)

Gradient Subvector n of Worker C ($n = 1, 2, 3$)

Gradient subvector 2 summation for Workers B and C

Gradient subvector n summation for Workers A, B, and C ($n = 1, 2, 3$)

Gradient subvector n of Worker B ($n = 1, 2, 3$)

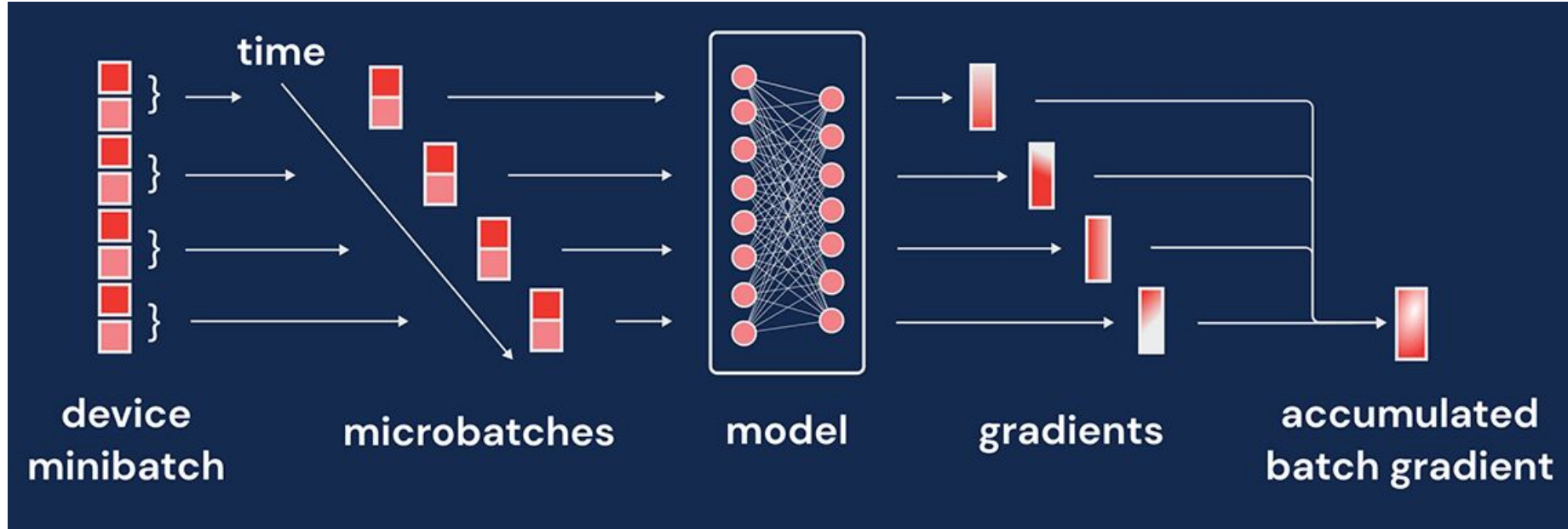
Gradient subvector 1 summation for Workers A and B

Gradient subvector 3 summation for Workers A and C

Sum of full gradient vectors of Workers A, B, and C

Gradient accumulation

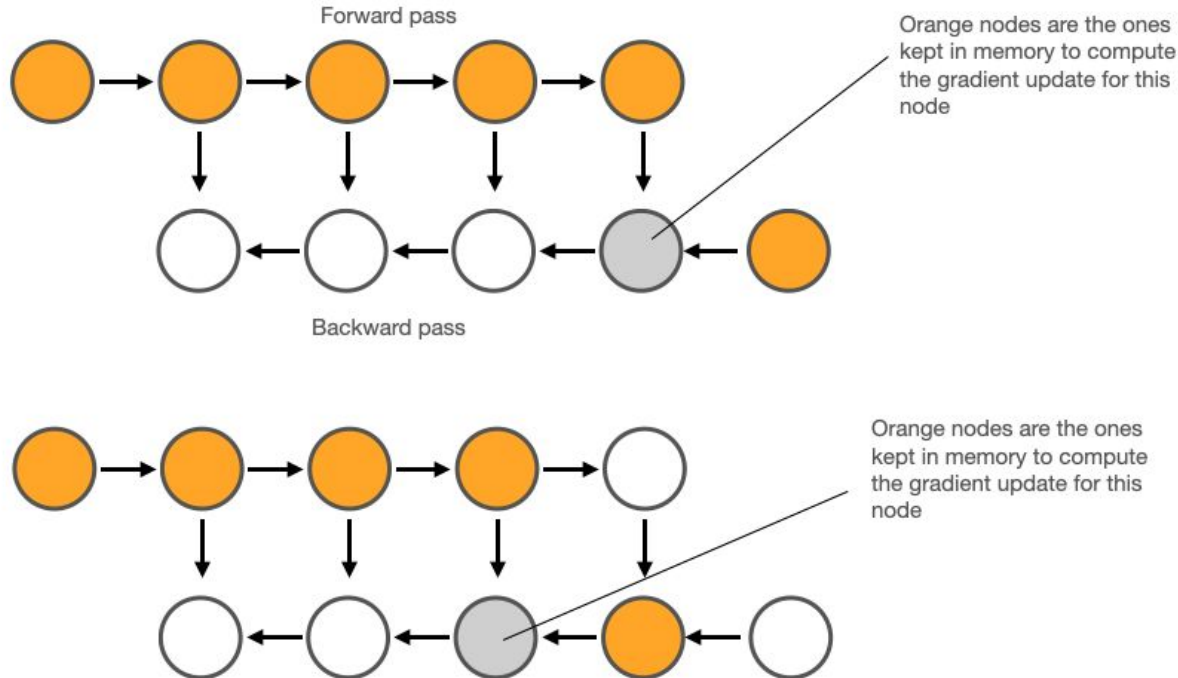
1. **Desired batch size**
does not fit in memory



Regular backpropagation

2. **Single object** does not fit in memory

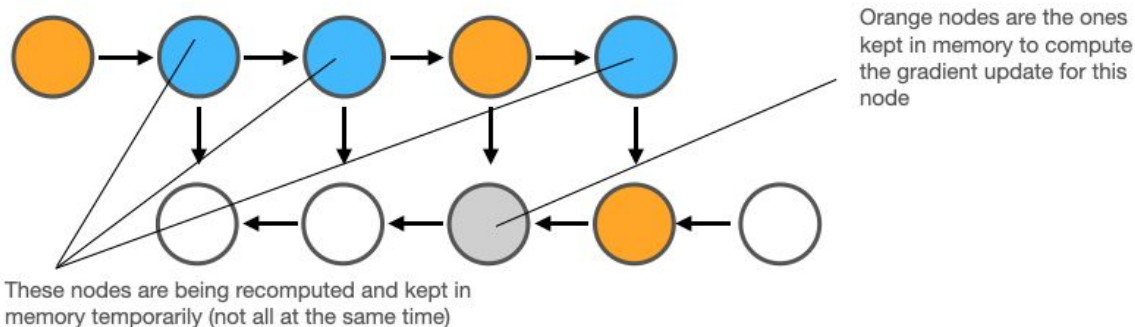
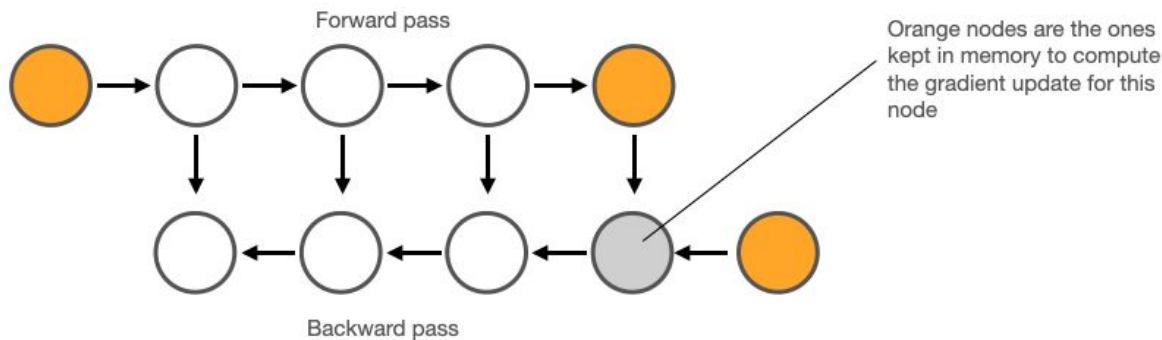
Drawing inspired by <https://github.com/cybertronai/gradient-checkpointing>



Low-memory backpropagation

2. **Single object** does not fit in memory

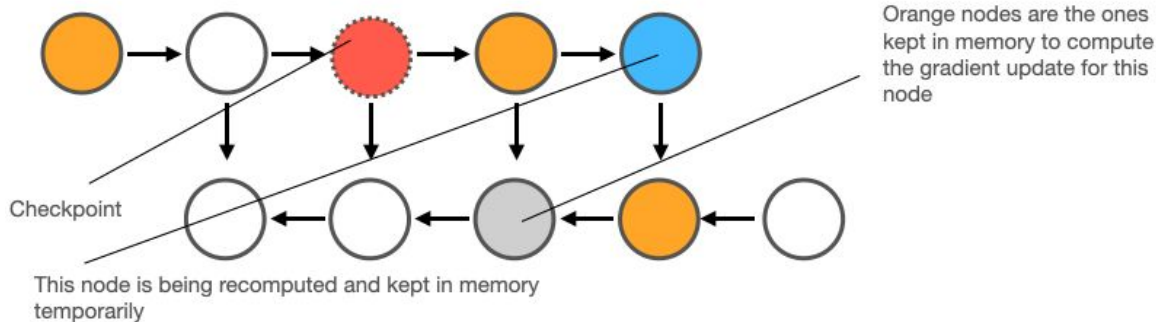
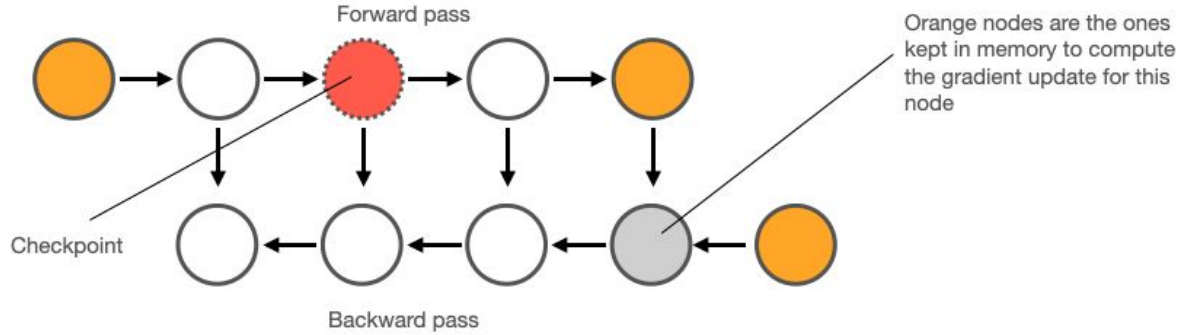
Drawing inspired by <https://github.com/cybertronai/gradient-checkpointing>



Gradient checkpointing

2. **Single object** does not fit in memory

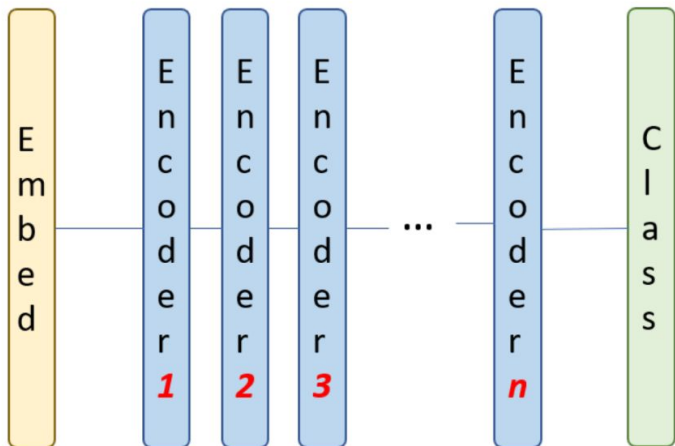
Drawing inspired by <https://github.com/cybertronai/gradient-checkpointing>



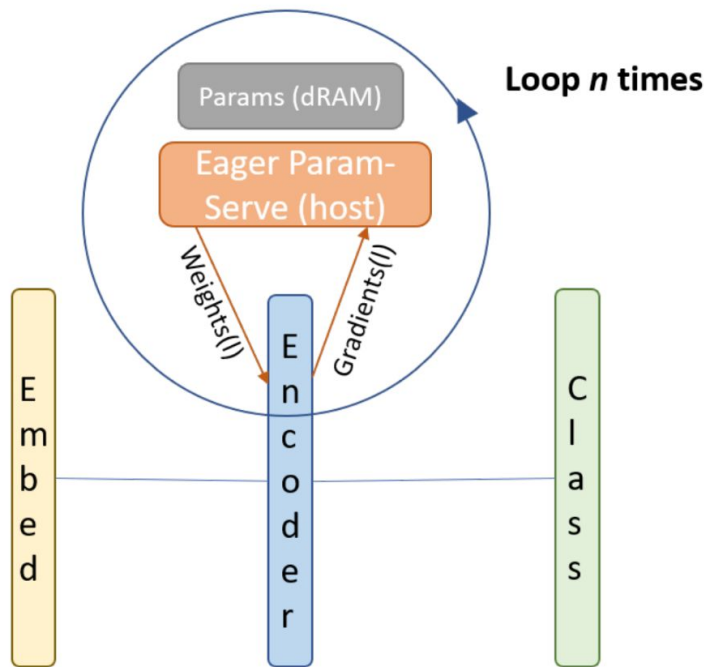
Memory offloading

3. Model parameters do not fit in memory

Conventional Execution n -layer NN

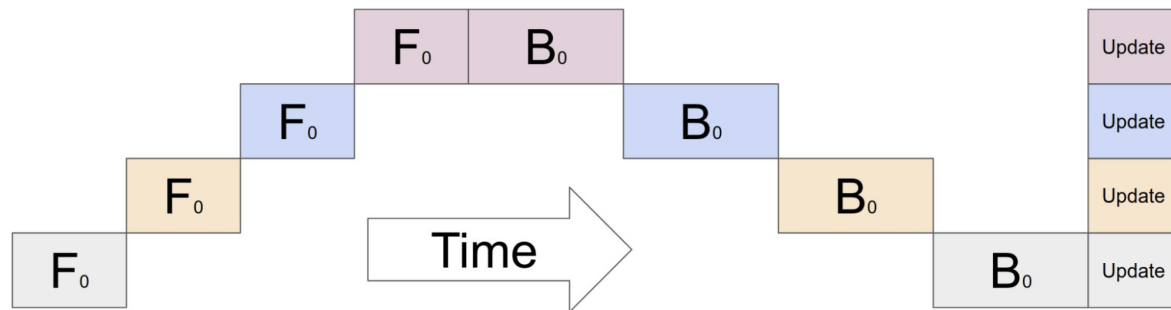
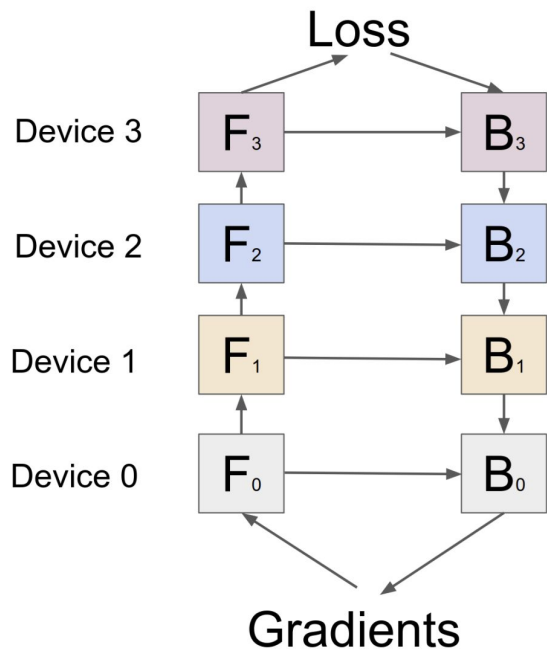


EPS with L2L execution



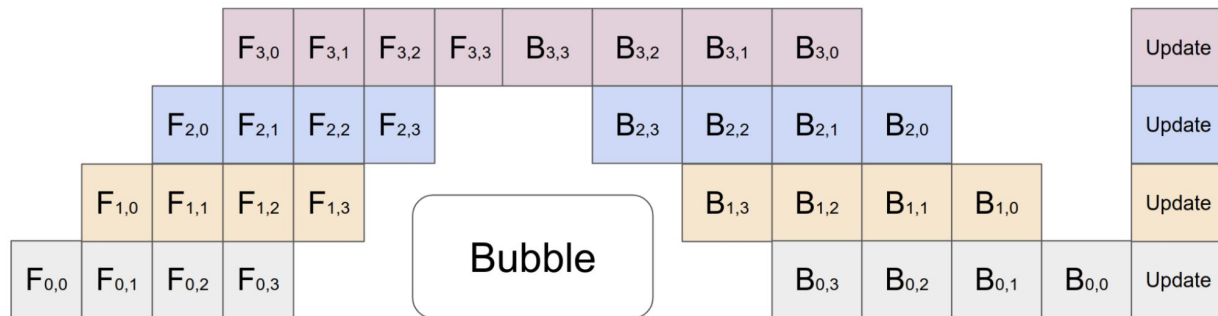
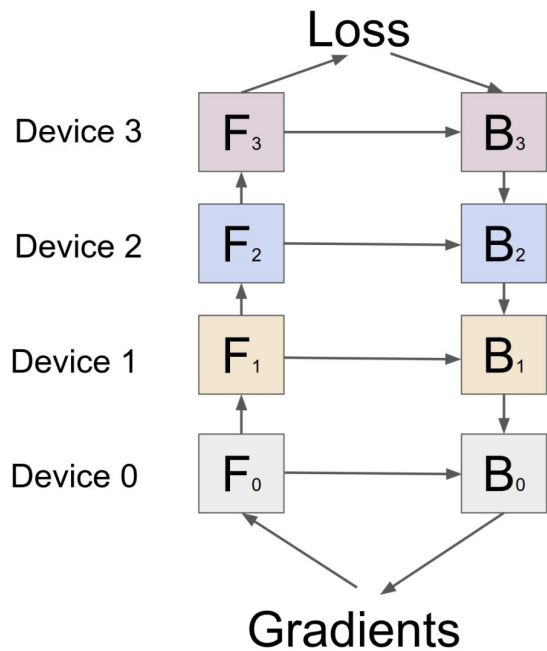
Model-parallel training

3. Model parameters do not fit in memory



Pipeline Parallelism

3. Model parameters do not fit in memory



Conclusion

- **AMP** accelerates training with a memory overhead
- **Data-parallel:** default choice if your model fits into 1 GPU
- **Model-parallel:** depends on model architecture but sometimes can be more effective